

Introduction to Machine Learning

Neural Networks Problems

Prof. Sundeep Rangan and Yao Wang

1. Consider a neural network on a 3-dimensional input $\mathbf{x} = (x_1, x_2, x_3)$ with weights and biases:

$$W^H = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}, \quad b^H = \begin{bmatrix} 0 \\ 0 \\ -1 \\ 1 \end{bmatrix} \quad W^O = [1, 1, -1, -1], \quad b^O = -1.5.$$

Assume the network uses the threshold activation function (??)

$$g_{\text{act}}(z) = \begin{cases} 1, & \text{if } z \geq 0 \\ 0, & \text{if } z < 0. \end{cases} \quad (1)$$

and the threshold output function (??):

$$\hat{y} = \begin{cases} 1 & z^O \geq 0 \\ 0 & z^O < 0. \end{cases} \quad (2)$$

- (a) Write the components of $\mathbf{z}^H$ and $\mathbf{u}^H$ as a function of (x_1, x_2, x_3) . For each component j , indicate where in the (x_1, x_2, x_3) hyperplane $u_j^H = 1$.

Solution: We have $\mathbf{z}^H = W^H \mathbf{x} + \mathbf{b}^H$:

$$\begin{aligned} z_1^H &= x_1 + x_3 \\ z_2^H &= x_2 + x_3 \\ z_3^H &= x_1 + x_2 - 1 \\ z_4^H &= x_1 + x_2 + x_3 + 1 \end{aligned}$$

Since $u_j^H = g_{\text{act}}(z_j^H)$, we have:

$$\begin{aligned} u_1^H &= \begin{cases} 1 & \text{if } x_1 + x_3 \geq 0 \\ 0 & \text{otherwise} \end{cases} \\ u_2^H &= \begin{cases} 1 & \text{if } x_2 + x_3 \geq 0 \\ 0 & \text{otherwise} \end{cases} \\ u_3^H &= \begin{cases} 1 & \text{if } x_1 + x_2 \geq 1 \\ 0 & \text{otherwise} \end{cases} \\ u_4^H &= \begin{cases} 1 & \text{if } x_1 + x_2 + x_3 \geq -1 \\ 0 & \text{otherwise} \end{cases} \end{aligned}$$

The hyperplanes where $u_j^H = 1$ are:

- $u_1^H = 1$: $x_1 + x_3 \geq 0$
- $u_2^H = 1$: $x_2 + x_3 \geq 0$
- $u_3^H = 1$: $x_1 + x_2 \geq 1$
- $u_4^H = 1$: $x_1 + x_2 + x_3 \geq -1$

- (b) Write z^O as a function of (x_1, x_2, x_3) . In what region is $\hat{y} = 1$? (You can describe in mathematical formulae).

Solution: We have $z^O = W^O \mathbf{u}^H + b^O = u_1^H + u_2^H - u_3^H - u_4^H - 1.5$.

Since each $u_j^H \in \{0, 1\}$, z^O can take values in $\{-3.5, -2.5, -1.5, -0.5, 0.5, 1.5, 2.5, 3.5\}$.

$\hat{y} = 1$ when $z^O \geq 0$, which occurs when:

$$u_1^H + u_2^H - u_3^H - u_4^H \geq 1.5$$

Since the left side is an integer, this means:

$$u_1^H + u_2^H - u_3^H - u_4^H \geq 2$$

This is satisfied when:

- $u_1^H = u_2^H = u_4^H = 1$ and $u_3^H = 0$, i.e., $x_1 + x_3 \geq 0$, $x_2 + x_3 \geq 0$, $x_1 + x_2 < 1$, and $x_1 + x_2 + x_3 \geq -1$
- $u_1^H = u_2^H = u_3^H = u_4^H = 1$, i.e., $x_1 + x_3 \geq 0$, $x_2 + x_3 \geq 0$, $x_1 + x_2 \geq 1$, and $x_1 + x_2 + x_3 \geq -1$

The feasible regions are:

$$\begin{aligned} \hat{y} = 1 \text{ when: } & (x_1 + x_3 \geq 0) \wedge (x_2 + x_3 \geq 0) \wedge (x_1 + x_2 < 1) \wedge (x_1 + x_2 + x_3 \geq -1) \\ & \text{OR } (x_1 + x_3 \geq 0) \wedge (x_2 + x_3 \geq 0) \wedge (x_1 + x_2 \geq 1) \wedge (x_1 + x_2 + x_3 \geq -1) \end{aligned}$$

2. Consider a neural network used for regression with a scalar input x and scalar target y ,

$$\begin{aligned} z_j^H &= W_j^H x + b_j^H, \quad u_j^H = \max\{0, z_j^H\}, \quad j = 1, \dots, N_h \\ z^O &= \sum_{k=1}^{N_h} W_k^O u_k^H + b^O, \quad \hat{y} = g_{\text{out}}(z^O). \end{aligned}$$

The hidden weights and biases are:

$$\mathbf{W}^H = \begin{bmatrix} -1 \\ 1 \\ 1 \end{bmatrix}, \quad \mathbf{b}^H = \begin{bmatrix} -1 \\ 1 \\ -2 \end{bmatrix}.$$

- (a) What is the number N_h of hidden units? For each $j = 1, \dots, N_h$, draw u_j^H as a function of x over some suitable range of values x . You may draw them on one plot, or on multiple plots.

Solution: $N_h = 3$ (from the dimension of $\mathbf{W}^H$).

For each j :

$$u_1^H(x) = \max\{0, -x - 1\} = \begin{cases} -x - 1 & \text{if } x \leq -1 \\ 0 & \text{if } x > -1 \end{cases}$$

$$u_2^H(x) = \max\{0, x + 1\} = \begin{cases} 0 & \text{if } x < -1 \\ x + 1 & \text{if } x \geq -1 \end{cases}$$

$$u_3^H(x) = \max\{0, x - 2\} = \begin{cases} 0 & \text{if } x < 2 \\ x - 2 & \text{if } x \geq 2 \end{cases}$$

These are ReLU functions with breakpoints at $x = -1$, $x = -1$, and $x = 2$ respectively.

- (b) Since the network is for regression, you may choose the activation function $g_{\text{out}}(z^o)$ to be linear. Given training data (x_i, y_i) , $i = 1, \dots, N$, formally define the loss function you would use to train the network?

Solution: For regression with linear output activation, we use the mean squared error (MSE) loss:

$$L = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 = \frac{1}{N} \sum_{i=1}^N \left(y_i - \left(\sum_{k=1}^{N_h} W_k^o u_k^H(x_i) + b^o \right) \right)^2$$

- (c) Using the output activation and loss function selected in part (b), set up the formulation to determine output weights and bias, $\mathbf{W}^o$, b^o , for the training data below. You should be able to find a closed form solution. Write a few line of python code to solve the problem.

x_i	-2	-1	0	3	3.5
y_i	0	0	1	3	3

Solution: First, compute $\mathbf{u}^H(x_i)$ for each training point:

$$\mathbf{u}^H(-2) = [1, 0, 0]^T$$

$$\mathbf{u}^H(-1) = [0, 0, 0]^T$$

$$\mathbf{u}^H(0) = [0, 1, 0]^T$$

$$\mathbf{u}^H(3) = [0, 4, 1]^T$$

$$\mathbf{u}^H(3.5) = [0, 4.5, 1.5]^T$$

Let $\mathbf{U} = [\mathbf{u}^H(x_1), \dots, \mathbf{u}^H(x_5)]^T$ (5×3 matrix) and $\mathbf{y} = [0, 0, 1, 3, 3]^T$. We solve: $\mathbf{y} = \mathbf{U}\mathbf{W}^o + b^o\mathbf{1}$, where $\mathbf{1}$ is a vector of ones. This is a linear system: $[\mathbf{U}, \mathbf{1}][\mathbf{W}^o; b^o] = \mathbf{y}$.

```
import numpy as np

# Training data
x = np.array([-2, -1, 0, 3, 3.5])
y = np.array([0, 0, 1, 3, 3])

# Compute hidden layer outputs
```

```

W_hid = np.array([[ -1], [ 1], [ 1]])
b_hid = np.array([[ -1], [ 1], [ -2]])
u_hid = np.maximum(0, W_hid @ x.reshape(1, -1) + b_hid).T

# Set up linear system: [U, ones] @ [W_out; b_out] = y
U = np.hstack([u_hid, np.ones((5, 1))])
params = np.linalg.lstsq(U, y, rcond=None)[0]
W_out = params[:3]
b_out = params[3]

print(f"W_out = {W_out}")
print(f"b_out = {b_out}")

```

- (d) Based on your solution for the output weights and bias, draw $\hat{y}$ vs. x over some suitable range of values x . Write a few line of python code to do this.

Solution:

```

import numpy as np
import matplotlib.pyplot as plt

# Parameters (from previous solution)
W_hid = np.array([[ -1], [ 1], [ 1]])
b_hid = np.array([[ -1], [ 1], [ -2]])
W_out = np.array([0, 0.5, 1]) # Example solution
b_out = 0.5 # Example solution

# Evaluate over range
x_plot = np.linspace(-3, 5, 1000)
u_hid_plot = np.maximum(0, W_hid @ x_plot.reshape(1, -1) + b_hid)
y_hat = W_out @ u_hid_plot + b_out

plt.plot(x_plot, y_hat)
plt.xlabel('x')
plt.ylabel('$\\hat{y}$')
plt.title('Network output vs input')
plt.grid(True)
plt.show()

```

- (e) Write a function predict to output $\hat{y}$ given a vector of inputs x . Assume x is a vector representing a batch of samples and y_{hat} is a vector with the corresponding outputs. Use the activation function you selected in part (b), but your function should take the weights and biases for both layers as inputs. Clearly state any assumptions on the formats for the weights and biases. Also, to receive full credit, you must not use any for loops.

Solution: Assumptions:

- x is a 1D array of shape $(N,)$ where N is the batch size
- W_{hid} is a 1D array of shape $(N_h,)$

- `b_hid` is a 1D array of shape $(N_h,)$
- `W_out` is a 1D array of shape $(N_h,)$
- `b_out` is a scalar

```
import numpy as np

def predict(x, W_hid, b_hid, W_out, b_out):
    """
    Predict output for batch of inputs.

    Args:
        x: 1D array of shape (N,) — batch of inputs
        W_hid: 1D array of shape (N_h,) — hidden layer weights
        b_hid: 1D array of shape (N_h,) — hidden layer biases
        W_out: 1D array of shape (N_h,) — output layer weights
        b_out: scalar — output layer bias

    Returns:
        yhat: 1D array of shape (N,) — predictions
    """
    # Hidden layer: z_hid shape (N_h, N)
    z_hid = W_hid[:, np.newaxis] * x[np.newaxis, :] + b_hid[:, np.newaxis]
    u_hid = np.maximum(0, z_hid) # ReLU activation, shape (N_h, N)

    # Output layer: linear
    yhat = W_out @ u_hid + b_out # shape (N,)

    return yhat
```

3. Consider a neural network that takes each input $\mathbf{x}$ and produces a prediction $\hat{y}$ given

$$\begin{aligned} z_j &= \sum_{k=1}^{N_i} W_{jk} x_k + b_j, \quad u_j = 1/(1 + \exp(-z_j)), \quad j = 1, \dots, M, \\ \hat{y} &= \frac{\sum_{j=1}^M a_j u_j}{\sum_{j=1}^M u_j}, \end{aligned} \quad (3)$$

where M is the number of hidden units and is fixed (i.e. not trainable). To train the model, we get training data $(\mathbf{x}_i, y_i)$, $i = 1, \dots, N$.

- (a) Rewrite the equations (??) for the batch of inputs $\mathbf{x}_i$ from the training data. That is, correctly add the indexing i , to the equations.

Solution:

$$z_{ij} = \sum_{k=1}^{N_i} W_{jk} x_{ik} + b_j, \quad u_{ij} = \frac{1}{1 + \exp(-z_{ij})}, \quad j = 1, \dots, M, \quad i = 1, \dots, N \quad (4)$$

$$\hat{y}_i = \frac{\sum_{j=1}^M a_j u_{ij}}{\sum_{j=1}^M u_{ij}}, \quad i = 1, \dots, N \quad (5)$$

- (b) If we use a loss function,

$$L = \sum_{i=1}^N (y_i - \hat{y}_i)^2,$$

draw the computation graph describing the mapping from $\mathbf{x}_i$ and parameters to L . Indicate which nodes are trainable parameters.

Solution: The computation graph is:

$$\begin{aligned} \mathbf{x}_i &\rightarrow \mathbf{z}_i = \mathbf{W}\mathbf{x}_i + \mathbf{b} \quad (\text{trainable: } \mathbf{W}, \mathbf{b}) \\ \mathbf{z}_i &\rightarrow \mathbf{u}_i = \sigma(\mathbf{z}_i) \quad (\text{sigmoid}) \\ \mathbf{u}_i &\rightarrow \hat{y}_i = \frac{\sum_{j=1}^M a_j u_{ij}}{\sum_{j=1}^M u_{ij}} \quad (\text{weighted average}) \\ \hat{y}_i, y_i &\rightarrow L_i = (y_i - \hat{y}_i)^2 \\ L_i &\rightarrow L = \sum_{i=1}^N L_i \end{aligned}$$

Trainable parameters: W_{jk} , b_j (and a_j if trainable, but stated as fixed).

- (c) Compute the gradient $\partial L / \partial \hat{y}_i$ for all i .

Solution:

$$\frac{\partial L}{\partial \hat{y}_i} = \frac{\partial}{\partial \hat{y}_i} \sum_{k=1}^N (y_k - \hat{y}_k)^2 = -2(y_i - \hat{y}_i)$$

So $\partial L / \partial \hat{\mathbf{y}} = -2(\mathbf{y} - \hat{\mathbf{y}})$ where $\hat{\mathbf{y}} = [\hat{y}_1, \dots, \hat{y}_N]^T$.

- (d) Suppose that, in backpropagation, we have computed $\partial L/\partial \hat{y}_i$ for all i , represented as $\partial L/\partial \hat{\mathbf{y}}$. Describe how to compute the components of the gradient $\partial L/\partial \mathbf{u}$.

Solution: We have $\hat{y}_i = \frac{\sum_{j=1}^M a_j u_{ij}}{S_i}$ where $S_i = \sum_{j=1}^M u_{ij}$.

Using the quotient rule:

$$\frac{\partial \hat{y}_i}{\partial u_{ij}} = \frac{a_j S_i - \sum_{k=1}^M a_k u_{ik}}{S_i^2} = \frac{a_j - \hat{y}_i}{S_i}$$

By the chain rule:

$$\frac{\partial L}{\partial u_{ij}} = \frac{\partial L}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial u_{ij}} = \frac{\partial L}{\partial \hat{y}_i} \frac{a_j - \hat{y}_i}{S_i}$$

In matrix form, if $\partial L/\partial \hat{\mathbf{y}}$ is a vector of length N and $\mathbf{U}$ is $N \times M$:

$$\frac{\partial L}{\partial \mathbf{U}} = \left(\frac{\partial L}{\partial \hat{\mathbf{y}}} \odot \frac{1}{\mathbf{S}} \right) (\mathbf{a} - \hat{\mathbf{y}})^T$$

where $\odot$ is element-wise multiplication, $\mathbf{S} = [S_1, \dots, S_N]^T$, and the outer product gives the $N \times M$ gradient matrix.

- (e) Suppose that we have computed the gradient $\partial L/\partial \mathbf{u}$, describe how would you compute the gradient $\partial L/\partial \mathbf{z}$.

Solution: Since $u_{ij} = \sigma(z_{ij}) = 1/(1 + \exp(-z_{ij}))$, we have:

$$\frac{\partial u_{ij}}{\partial z_{ij}} = \sigma(z_{ij})(1 - \sigma(z_{ij})) = u_{ij}(1 - u_{ij})$$

By the chain rule:

$$\frac{\partial L}{\partial z_{ij}} = \frac{\partial L}{\partial u_{ij}} \frac{\partial u_{ij}}{\partial z_{ij}} = \frac{\partial L}{\partial u_{ij}} u_{ij}(1 - u_{ij})$$

In matrix form:

$$\frac{\partial L}{\partial \mathbf{Z}} = \frac{\partial L}{\partial \mathbf{U}} \odot \mathbf{U} \odot (1 - \mathbf{U})$$

where all operations are element-wise.

- (f) Suppose that we have computed the gradient $\partial L/\partial \mathbf{z}$, describe how would you compute the gradient $\partial L/\partial W_{jk}$ and $\partial L/\partial b_j$.

Solution: Since $z_{ij} = \sum_{k=1}^{N_i} W_{jk} x_{ik} + b_j$, we have:

$$\frac{\partial z_{ij}}{\partial W_{jk}} = x_{ik}, \quad \frac{\partial z_{ij}}{\partial b_j} = 1$$

By the chain rule:

$$\begin{aligned} \frac{\partial L}{\partial W_{jk}} &= \sum_{i=1}^N \frac{\partial L}{\partial z_{ij}} \frac{\partial z_{ij}}{\partial W_{jk}} = \sum_{i=1}^N \frac{\partial L}{\partial z_{ij}} x_{ik} \\ \frac{\partial L}{\partial b_j} &= \sum_{i=1}^N \frac{\partial L}{\partial z_{ij}} \frac{\partial z_{ij}}{\partial b_j} = \sum_{i=1}^N \frac{\partial L}{\partial z_{ij}} \end{aligned}$$

In matrix form:

$$\frac{\partial L}{\partial \mathbf{W}} = \left(\frac{\partial L}{\partial \mathbf{Z}} \right)^T \mathbf{X}, \quad \frac{\partial L}{\partial \mathbf{b}} = \left(\frac{\partial L}{\partial \mathbf{Z}} \right)^T \mathbf{1}$$

where $\mathbf{X}$ is $N \times N_i$ with rows $\mathbf{x}_i^T$, and $\mathbf{1}$ is a vector of ones.

- (g) Put all above together, describe how would you compute the gradient $\partial L / \partial W_{jk}$ and $\partial L / \partial b_j$.

Solution: Backpropagation steps:

- i. Forward pass: Compute $\mathbf{z}_i$, $\mathbf{u}_i$, $\hat{y}_i$ for all i .
 - ii. Compute $\frac{\partial L}{\partial \hat{y}_i} = -2(y_i - \hat{y}_i)$ for all i .
 - iii. Compute $\frac{\partial L}{\partial u_{ij}} = \frac{\partial L}{\partial \hat{y}_i} \frac{a_j - \hat{y}_i}{S_i}$.
 - iv. Compute $\frac{\partial L}{\partial z_{ij}} = \frac{\partial L}{\partial u_{ij}} u_{ij}(1 - u_{ij})$.
 - v. Compute $\frac{\partial L}{\partial W_{jk}} = \sum_{i=1}^N \frac{\partial L}{\partial z_{ij}} x_{ik}$ and $\frac{\partial L}{\partial b_j} = \sum_{i=1}^N \frac{\partial L}{\partial z_{ij}}$.
- (h) Write a few lines of python code to implement the gradients $\partial L / \partial \mathbf{u}$ (as in part (d)), given the gradient $\partial L / \partial \hat{\mathbf{y}}$. Indicate how you represent the gradients. Full credit requires that you avoid for-loops.

Solution: Representation:

- $\partial L / \partial \hat{\mathbf{y}}$: 1D array of shape $(N,)$
- $\partial L / \partial \mathbf{U}$: 2D array of shape (N, M)
- $\mathbf{a}$: 1D array of shape $(M,)$
- $\hat{\mathbf{y}}$: 1D array of shape $(N,)$
- $\mathbf{U}$: 2D array of shape (N, M)

```
import numpy as np
```

```
def grad_L_wrt_u(dL_dyhat, a, yhat, U):
```

```
    """
```

```
    Compute gradient of L with respect to U.
```

```
    Args:
```

```
        dL_dyhat: 1D array of shape (N,) — dL/dyhat
```

```
        a: 1D array of shape (M,) — weights a_j
```

```
        yhat: 1D array of shape (N,) — predictions yhat_i
```

```
        U: 2D array of shape (N, M) — hidden activations u_{ij}
```

```
    Returns:
```

```
        dL_dU: 2D array of shape (N, M) — dL/dU
```

```
    """
```

```
    S = np.sum(U, axis=1) # S_i = sum_j u_{ij}, shape (N,)
```

```
    dL_dU = (dL_dyhat[:, np.newaxis] / S[:, np.newaxis]) * (a[np.newaxis, :] - yhat[:, np.newaxis])
```

```
    return dL_dU
```